# Advanced Empirical Research Methods Hilary 2025/26

Assessed Problem Set: Empirical IO

Candidate ID: 1091806

2026-04-13

## Contents

**Libraries** **2**

**Q1. Beer demand** **3**

Q1 (a) Data Description . . . 3

Q1 (b) Nested Logit Estimation . . . 6

Q1 (c) Elasticities, Marginal Costs and Diversion Ratios . . . 11

Q1 (d): £10 Tax per Unit of Alcohol Strength . . . 15

**Q2. Auto demand** **21**

Data Visualisation . . . 21

Q2 (a): Estimating Two BLP Specifications . . . 24

(a) . . . 31

(b) . . . 31

(c) . . . 31

## Libraries

```r
# Part 1 Empirical IO

# Beer demand

  library(tidyverse)
  library(ivreg)
  library(modelsummary)
  library(knitr)

# Auto demand

  library(BLPestimatoR)
  library(nleqslv)
```

Set up:

```r
# Set seed to 130

  set.seed(130)
```

# Q1. Beer demand

## Q1 (a) Data Description

```r
# Load the beer data

  beer <- read_csv("beer_data.csv")
```

```
## Rows: 43979 Columns: 8
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (8): product_id, market_id, num_prod_mkt, marketsize, quantity, price, m...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Preview structure

  glimpse(beer)
```

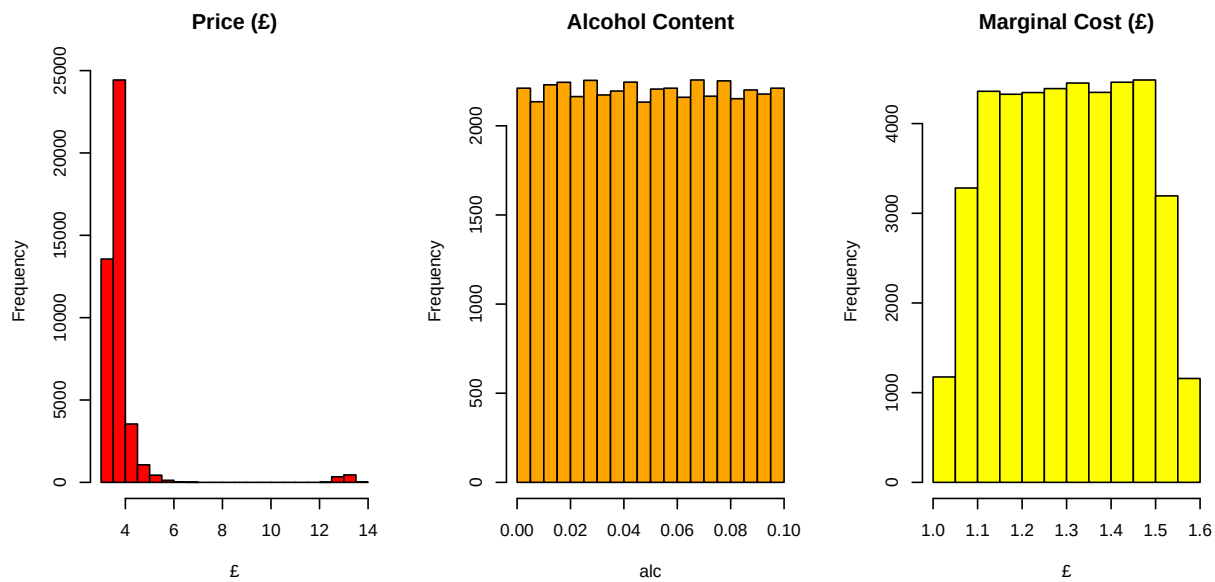
```
## Rows: 43,979
## Columns: 8
## $ product_id    <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1~
## $ market_id     <dbl> 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3,~
## $ num_prod_mkt  <dbl> 5, 5, 5, 5, 5, 8, 8, 8, 8, 8, 8, 8, 8, 7, 7, 7, 7, 7, 7,~
## $ marketsize    <dbl> 9502.325, 9502.325, 9502.325, 9502.325, 9502.325, 2723.8~
## $ quantity      <dbl> 262.04402, 1416.44809, 736.14570, 575.80224, 787.00124, ~
## $ price         <dbl> 3.588417, 4.279416, 3.701617, 3.839169, 3.698343, 3.3214~
## $ marginal_cost <dbl> 1.458464, 1.295263, 1.287914, 1.529528, 1.249649, 1.0918~
## $ alc_j         <dbl> 0.077651380, 0.051817406, 0.065659550, 0.040277764, 0.09~
```

```r
# Summary statistics for key variables

  beer %>%
    select(price, quantity, marginal_cost, alc_j, marketsize, num_prod_mkt) %>%
    summary()
```

```
##      price           quantity         marginal_cost       alc_j
##  Min.   : 3.072   Min.   :1.359e-02   Min.   :1.001   Min.   :8.790e-07
##  1st Qu.: 3.461   1st Qu.:1.215e+02   1st Qu.:1.176   1st Qu.:2.502e-02
##  Median : 3.614   Median :2.508e+02   Median :1.301   Median :5.000e-02
##  Mean   : 3.848   Mean   :3.280e+02   Mean   :1.300   Mean   :4.998e-02
##  3rd Qu.: 3.802   3rd Qu.:4.485e+02   3rd Qu.:1.425   3rd Qu.:7.500e-02
##  Max.   :13.621   Max.   :2.073e+03   Max.   :1.598   Max.   :1.000e-01
##    marketsize         num_prod_mkt
##  Min.   :   0.1579   Min.   : 1.000
##  1st Qu.:2526.2586   1st Qu.: 5.000
##  Median :4950.8309   Median : 7.000
##  Mean   :5001.0880   Mean   : 6.979
##  3rd Qu.:7517.4464   3rd Qu.: 9.000
```

```
##  Max.   :9999.1389   Max.   :10.000
```

```r
# Visualise distributions of price and alcohol content

  par(mfrow = c(1, 3))

  hist(beer$price,        main = "Price (£)",         xlab = "£",   col = "red")
  hist(beer$alc_j,        main = "Alcohol Content",   xlab = "alc", col = "orange")
  hist(beer$marginal_cost,main = "Marginal Cost (£)", xlab = "£",   col = "yellow")
```




```r
# Number of markets and products per market
cat("Number of unique markets:  ", n_distinct(beer$market_id), "\n")
```

```
## Number of unique markets:   8000
```

```r
cat("Number of unique products: ", n_distinct(beer$product_id), "\n")
```

```
## Number of unique products:  43979
```

```r
# Distribution of products per market

beer %>%
  group_by(market_id) %>%
  summarise(n_products = n()) %>%
  summary()
```

```
##    market_id      n_products
##  Min.   :   1   Min.   : 1.000
##  1st Qu.:2001   1st Qu.: 3.000
##  Median :4000   Median : 6.000
##  Mean   :4000   Mean   : 5.497
##  3rd Qu.:6000   3rd Qu.: 8.000
```

```
##  Max.   :8000   Max.   :10.000
```

**Comment:**

The data contains 8000 markets and 43979 product-market observations, with between 1 and 10 products per market (mean 7). Prices range from £3.07 to £13.62 with a mean of £3.85, and are right-skewed — most beers are cheap with a few expensive outliers. Marginal costs range from £1 to £1.6, always below price, consistent with positive markups. Alcohol content (`alc_j`) is uniformly distributed between 0 and 0.1, suggesting products are evenly spread across the strength spectrum. Market sizes range from 0 to 9999 potential purchase occasions.

## Q1 (b) Nested Logit Estimation

### Step 1: Compute Market Shares

```r
beer <- beer |>
  group_by(market_id) |>
  mutate(

    # Market share of product j in market t
    share_jt  = quantity / marketsize,

    # Aggregate share of all inside goods (all beers) in market t
    share_agg = sum(share_jt),

    # Outside good share: fraction who don't buy any beer
    share_0   = 1 - share_agg,

    # Within-group share: j's share among beer buyers
    # Since all inside goods are in one nest: s_{j|g} = s_jt / (1 - s_0t)
    share_jg  = share_jt / share_agg

  ) |>
  ungroup()

# Sanity checks
cat("Min outside good share: ", round(min(beer$share_0), 4), "\n")
```

```
## Min outside good share:  0.555
```

```r
cat("Max outside good share: ", round(max(beer$share_0), 4), "\n")
```

```
## Max outside good share:  0.8918
```

```r
cat("All shares positive:    ", all(beer$share_jt > 0), "\n")
```

```
## All shares positive:     TRUE
```

### Step 2: Construct the Dependent Variable and Regressors

```r
beer <- beer |>
  mutate(

    # Nested logit dependent variable: ln(s_jt) - ln(s_0t)
    delta     = log(share_jt) - log(share_0),

    # Within-group share term that appears on RHS
    ln_share_jg = log(share_jg)
  )

summary(beer$delta)
```

| Statistic | Value |
|---|---|
| Min | -4.609 |
| 1st Qu. | -2.73 |
| Median | -2.371 |
| Mean | -2.379 |
| 3rd Qu. | -2.001 |
| Max | -0.994 |

**Step 3: Estimate the Nested Logit Model**

The estimating equation is:

$$\ln(s_{jt}) - \ln(s_{0t}) = \beta_0 + \beta_{\text{alc}} \cdot \text{alc}_j - \alpha \cdot p_{jt} + \sigma \cdot \ln(s_{j|g,t}) + \xi_{jt}$$

Both `price` and `ln_share_jg` are **endogenous**:

- **Price** is endogenous because firms observe $\xi_{jt}$ before setting prices — beers with high unobserved quality command higher prices, creating a positive correlation between `price` and $\xi_{jt}$. OLS would therefore underestimate price sensitivity.
- **Within-group share** $\ln(s_{j|g,t})$ is endogenous because it depends on all prices in the market, which all depend on the $\xi$'s. If product $j$ has high $\xi_{jt}$, consumers substitute towards it, inflating its within-group share. OLS would therefore overestimate $\hat{\sigma}$.

So we need instruments:

- `marginal_cost` for `price` (cost shifter, excluded from demand)
- `num_prod_mkt` for `ln_share_jg` (more products dilutes within-group share but does not enter consumer utility directly)

```r
# OLS (for comparison - biased due to endogeneity)

  model_ols <- lm(delta ~ alc_j + price + ln_share_jg, data = beer)

# IV / 2SLS: instrument price with marginal_cost,
#            instrument ln_share_jg with num_prod_mkt

  model_iv <- ivreg(
    delta ~ alc_j + price + ln_share_jg |
            alc_j + marginal_cost + num_prod_mkt,
    data = beer
  )
```

```r
# Compare OLS and IV results

modelsummary(
  list("OLS" = model_ols, "IV (2SLS)" = model_iv),
  stars  = TRUE,
  gof_omit = "AIC|BIC|Log|F",
```

Table 2: Nested Logit Estimates

| | OLS | IV (2SLS) |
|---|---|---|
| (Intercept) | −0.166*** | −0.514*** |
| | (0.003) | (0.016) |
| alc_j | 0.707*** | 1.019*** |
| | (0.013) | (0.018) |
| price | −0.125*** | −0.098*** |
| | (0.000) | (0.002) |
| ln_share_jg | 0.916*** | 0.798*** |
| | (0.001) | (0.003) |
| Num.Obs. | 43 979 | 43 979 |
| R2 | 0.976 | 0.962 |
| R2 Adj. | 0.976 | 0.962 |
| RMSE | 0.08 | 0.10 |

+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

```
  title  = "Nested Logit Estimates"
)
```

```
# Check first-stage relevance: instruments must be correlated with endogenous vars

# First stage for price

  fs_price <- lm(price ~ alc_j + marginal_cost + num_prod_mkt, data = beer)

# First stage for ln_share_jg

  fs_share <- lm(ln_share_jg ~ alc_j + marginal_cost + num_prod_mkt, data = beer)
```

```
cat("=== First Stage: Price ===\n")
```

```
## === First Stage: Price ===
```

```
summary(fs_price)$coefficients
```

```
##                 Estimate  Std. Error     t value      Pr(>|t|)
## (Intercept)    4.4671611 0.050045115   89.262680  0.000000e+00
## alc_j          0.9454574 0.190038745    4.975077  6.546734e-07
## marginal_cost  0.8870845 0.037162736   23.870269 3.913577e-125
## num_prod_mkt  -0.2607295 0.002208348 -118.065416  0.000000e+00
```

```
cat("\n=== First Stage: ln(within-group share) ===\n")
```

```
##
```

```
## === First Stage: ln(within-group share) ===
```

```r
summary(fs_share)$coefficients
```

```
##                 Estimate   Std. Error    t value      Pr(>|t|)
## (Intercept)   -0.2983597 0.0183994562  -16.21568  5.797103e-59
## alc_j          3.4398668 0.0698691481   49.23299  0.000000e+00
## marginal_cost -0.3381204 0.0136631543  -24.74687 2.789482e-134
## num_prod_mkt  -0.1955449 0.0008119154 -240.84395  0.000000e+00
```

**Identification assumptions:** The validity of the IV estimates rests on two conditions.

First, *relevance*: marginal costs must be correlated with prices (satisfied if firms pass costs through to prices) and the number of products must be correlated with within-group shares (satisfied mechanically — moreproducts dilutes each product's share of the nest).

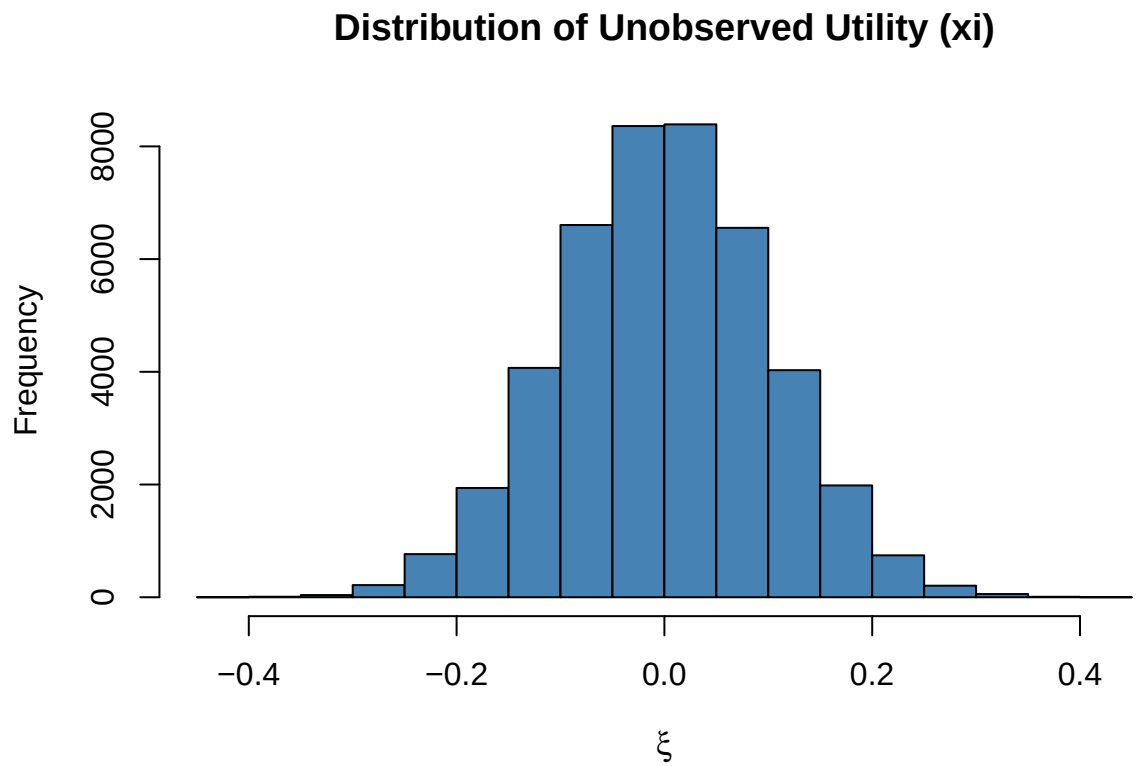
Second, *exclusion*: marginal costs must not directly affect consumer utility (reasonable — consumers care about price, not cost), and the number of products in the market must not directly shift a consumer's valuation of any individual beer (reasonable under the nested logit structure). The residuals $\xi_{jt}$ are our estimate of unobserved product quality — product attributes that consumers value but that we do not observe in the data.

**Step 4: Extract Residuals as Unobserved Utility**

```r
# Vector ξ

# Residuals from IV regression are our estimate of xi_{jt}
# These capture unobserved product quality not explained by alc_j or price
beer$xi <- residuals(model_iv)

summary(beer$xi)
```

```
##       Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
## -4.257e-01 -6.794e-02 -7.177e-05  0.000e+00  6.791e-02  4.144e-01
```

```r
hist(beer$xi,
     main = "Distribution of Unobserved Utility (xi)",
     xlab = expression(xi),
     col  = "steelblue",
     breaks = 30)
```




**Comment on estimates:** The IV price coefficient ($\hat{\alpha}$) is -0.098, which is negative as expected — higher prices reduce demand. The nesting parameter $\hat{\sigma}$ on `ln_share_jg` is 0.798, which lies in (0,1), consistent with the nested logit model. Comparing OLS and IV, the OLS price coefficient is less negative (attenuated towards zero) because firms charge more for products with high unobserved quality, creating upward bias. The instruments correct for this. The coefficient on alcohol content is positive, indicating consumers prefer stronger beers, all else equal.

## Q1 (c) Elasticities, Marginal Costs and Diversion Ratios

### Step 1: Extract Parameters and Compute Own-Price Elasticities

We extract the estimated price parameter $\hat{\alpha}$ and nesting parameter $\hat{\sigma}$ from the IV model estimated in part (b). Note we flip the sign on the price coefficient so that $\alpha$ is positive, matching the notation in the elasticity and markup formulas.

```r
# Extract estimated parameters from IV model

alpha <- -coef(model_iv)["price"]          # price parameter (positive)
sigma <- coef(model_iv)["ln_share_jg"]     # nesting parameter

cat("alpha:", round(alpha, 4), "\n")
```

```
## alpha: 0.0977
```

```r
cat("sigma:", round(sigma, 4), "\n")
```

```
## sigma: 0.7977
```

With $\hat{\alpha}$ and $\hat{\sigma}$ in hand, we can compute own-price elasticities for each product-market observation using the nested logit formula:

$$\eta_{jj} = -\alpha \cdot p_{jt} \cdot \frac{1 - \sigma s_{j|g,t} - (1-\sigma) s_{jt}}{1-\sigma}$$

The term in the numerator reflects that a price rise loses customers to two destinations — other beers (weighted by $\sigma$) and the outside good (weighted by $1-\sigma$).

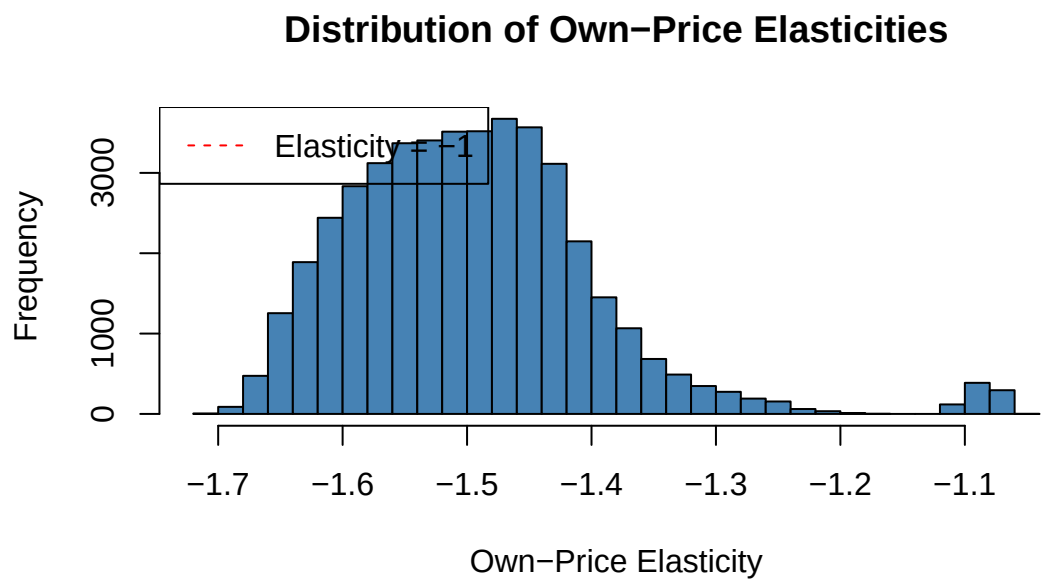
```r
# Own-price elasticity for nested logit (single nest):
# eta_jj = -alpha * p_jt * (1 - sigma*s_{j|g} - (1-sigma)*s_j) / (1 - sigma)

beer <- beer |>
  mutate(
    elast_own = -alpha * price *
      (1 - sigma * share_jg - (1 - sigma) * share_jt) / (1 - sigma)
  )
```

*Summary table*

| Statistic | Value |
|---|---|
| Min | -1.703 |
| 1st Qu. | -1.567 |
| Median | -1.502 |
| Mean | -1.495 |
| 3rd Qu. | -1.441 |
| Max | -1.06 |

To visualise the distribution and check how many products fall in the inelastic region ($|\eta| < 1$), we plot a histogram with a reference line at $-1$.

```r
hist(beer$elast_own,
     main = "Distribution of Own-Price Elasticities",
     xlab = "Own-Price Elasticity",
     col  = "steelblue",
     breaks = 40)
abline(v = -1, col = "red", lty = 2, lwd = 2)
legend("topleft", legend = "Elasticity = -1", col = "red", lty = 2)
```




**Step 2: Back Out Marginal Costs**

Under single-product Nash, the markup is:

$$\text{markup}_{jt} = p_{jt} - mc_{jt} = \frac{1-\sigma}{\alpha(1-\sigma s_{j|g,t} - (1-\sigma)s_{jt})}$$

```r
beer <- beer |>
  mutate(
    markup_implied = (1 - sigma) /
      (alpha * (1 - sigma * share_jg - (1 - sigma) * share_jt)),
    mc_implied = price - markup_implied
  )
```

*Summary of backed-out marginal costs*

| Statistic | Marginal Cost | Markup |
|---|---|---|
| Min | 0.73 | 2.095 |
| Median | 1.215 | 2.359 |
| Mean | 1.214 | 2.635 |

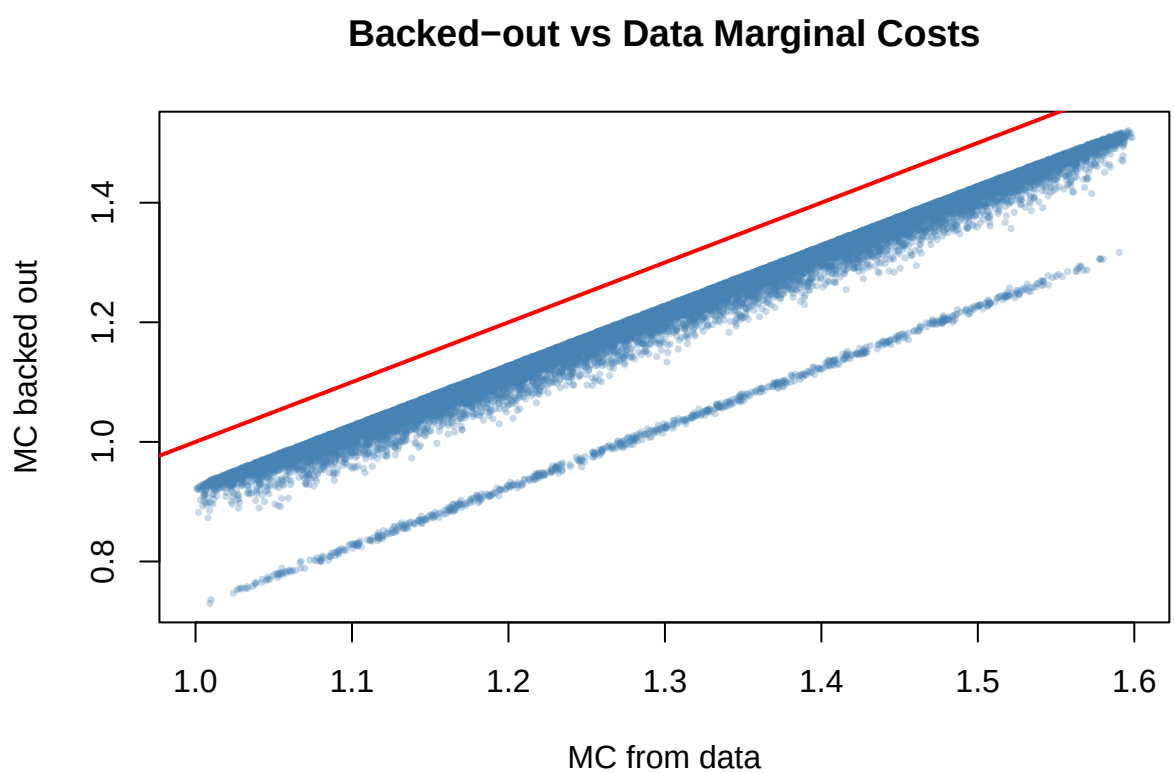
| Statistic | Marginal Cost | Markup |
|---|---|---|
| Max | 1.521 | 12.598 |

As a sanity check, we compare our backed-out marginal costs with the ones provided in the data. They should be strongly correlated if both the demand model and the single-product Nash assumption are reasonable approximations.

```
# Check: compare backed-out mc with mc in data

cat("Correlation between backed-out and data mc:",
    round(cor(beer$mc_implied, beer$marginal_cost), 4), "\n")
```

```
## Correlation between backed-out and data mc: 0.9829
```

```
plot(beer$marginal_cost, beer$mc_implied,
     main = "Backed-out vs Data Marginal Costs",
     xlab = "MC from data", ylab = "MC backed out",
     pch  = 16, col = adjustcolor("steelblue", alpha.f = 0.3),
     cex  = 0.5)
abline(0, 1, col = "red", lwd = 2)
```




### Step 3: Diversion Ratio to Outside Good

The diversion ratio to the outside good tells us what fraction of lost customers leave the beer market

entirely when a product's price rises marginally, shown as below:

$$DR_{j\to 0} = \frac{(1-\sigma)\cdot s_{0t}}{1-\sigma s_{j|g,t} - (1-\sigma)s_{jt}}$$

Comparing this to the raw outside good share illustrates the role of $\sigma$: in a simple logit ($\sigma = 0$), these would be equal; with $\sigma > 0$, the diversion ratio is lower because consumers prefer to stay within the nest.

```
beer <- beer |>
  mutate(
    diversion_outside = (1 - sigma) * share_0 /
      (1 - sigma * share_jg - (1 - sigma) * share_jt)
  )
```

*Compare diversion ratio to outside good share directly*

| Statistic | Diversion Ratio to Outside | Outside Good Share |
|---|---|---|
| Mean | 0.162 | 0.609 |
| Median | 0.138 | 0.598 |

**Comment:** Own-price elasticities are all negative and mostly elastic (below -1), as expected for individual beer products. Backed-out marginal costs are positive and broadly consistent with those in the data. The diversion ratio to the outside good (0.162) is lower than the outside good share (0.609), reflecting that when a beer's price rises, consumers tend to substitute towards other beers rather than exit the market — consistent with $\hat{\sigma} = 0.798 > 0$. This highlights a key flexibility limitation: the nested logit forces all within-nest cross-price elasticities to be identical, which may be unrealistic if beers differ substantially in their similarity to one another.

## Q1 (d): £10 Tax per Unit of Alcohol Strength

The government is considering a £10 tax per unit of alcohol strength (`alc_j`). This increases each product's marginal cost by $10 \times \text{alc}_j$ — so stronger beers face a larger cost shock. We use our backed-out marginal costs from part (c) as the pre-tax baseline, rather than the marginal costs given in the data, to ensure internal consistency with the demand model.

### Step 1: Compute New Marginal Costs After Tax

We first construct the post-tax marginal cost for each product.

```
tax_rate <- 10

beer <- beer |>
  mutate(
    # Tax increases mc by 10 * alc_j for each product
    mc_tax = mc_implied + tax_rate * alc_j,
    delta_mc = mc_tax - mc_implied
  )

cat("Mean marginal cost increase (delta_mc):",
    round(mean(beer$delta_mc), 4), "\n")
```

```
## Mean marginal cost increase (delta_mc): 0.4998
```

### Step 2: Define Share Function for Contraction Mapping

To solve for counterfactual equilibrium prices, we need a function that computes nested logit market shares given a price vector. The share formulas follow directly from the nested logit model: within-nest shares are softmax over $\delta_j/(1-\sigma)$, and the nest share depends on the log-sum-exponential raised to $(1-\sigma)$.

```
# Function to compute nested logit shares given prices in a single market

compute_shares <- function(price_t, delta_base_t, sigma) {

  # delta_base_t = beta0 + beta_alc*alc_j + xi_jt (no price term)
  # Full mean utility including price:

    delta_t <- delta_base_t - alpha * price_t   # using alpha from model_iv

  # Within-nest shares

    exp_d  <- exp(delta_t / (1 - sigma))
    sum_eg <- sum(exp_d)

    sj_g <- exp_d / sum_eg

  # Group share

    sg <- sum_eg^(1 - sigma) / (1 + sum_eg^(1 - sigma))
```

```r
  # Product shares

    sj <- sj_g * sg

  return(list(sj = sj, sj_g = sj_g, s0 = 1 - sum(sj)))
}
```

**Step 3: Contraction Mapping to Solve for Counterfactual Prices**

The counterfactual equilibrium cannot be solved analytically because prices and shares are jointly determined: each firm's price depends on shares, which depend on all prices in the market. We therefore use a contraction mapping. Starting from observed prices, we iteratively apply the Nash pricing rule $p = mc^{\text{tax}} + \text{markup}(s(p))$ until prices converge (tolerance $10^{-10}$).

A key subtlety: we need the "base utility"

$\delta_{jt}^{\text{base}} = \beta_0 + \beta_{\text{alc}}\text{alc}_j + \xi_{jt}$

that excludes the price term, which we recover from the estimated $\delta_{jt}$ by adding back $\alpha p_{jt}$ and subtracting the $\sigma \ln(s_{j|g,t})$ term.

```r
# We solve market by market
# For each market: iterate p = mc_tax + markup(shares(p)) until convergence

solve_prices <- function(beer_mkt, mc_col, sigma, alpha,
                         tol = 1e-10, maxit = 10000) {

  # Base utility (excludes price): fitted values + alpha*price removes price effect

    delta_base <- beer_mkt$delta - log(beer_mkt$share_jg) * sigma +
                  alpha * beer_mkt$price

  # Actually: delta_base = beta0 + beta_alc*alc_j + xi = delta - sigma*ln(s_jg) + alpha*p
  # Since delta = beta0 + beta_alc*alc_j - alpha*p + sigma*ln(s_jg) + xi
  # => delta_base = delta + alpha*p - sigma*ln(s_jg)

    delta_base <- beer_mkt$delta + alpha * beer_mkt$price -
                  sigma * log(beer_mkt$share_jg)


     mc_t  <- beer_mkt[[mc_col]]
     p_t   <- beer_mkt$price  # starting guess

  for (iter in 1:maxit) {

    shares <- compute_shares(p_t, delta_base, sigma)
    sj_t   <- shares$sj
    sj_g_t <- shares$sj_g
```

```r
    # Single-product Nash markup

     markup_t <- (1 - sigma) /
      (alpha * (1 - sigma * sj_g_t - (1 - sigma) * sj_t))

      p_new <- mc_t + markup_t

      if (max(abs(p_new - p_t)) < tol) break
      p_t <- p_new
    }

  return(p_new)
}
```

We loop over all 8000 markets, solving two price vectors in each: one with the observed marginal costs (a consistency check that should reproduce observed prices), and one with the post-tax marginal costs.

```r
# Solve for counterfactual prices in each market

  markets <- unique(beer$market_id)

# Pre-allocate

  beer$price_tax  <- NA
  beer$price_notax <- NA  # check: should recover original prices

  for (m in markets) {

    idx <- beer$market_id == m
    beer_m <- beer[idx, ]

    # Check: no tax should recover original prices
    beer$price_notax[idx] <- solve_prices(beer_m, "mc_implied", sigma, alpha)

    # With tax
    beer$price_tax[idx]   <- solve_prices(beer_m, "mc_tax",    sigma, alpha)
  }
```

### Step 4: Check — No-Tax Prices Should Equal Observed Prices

Before analysing tax effects, we verify that running our contraction mapping with unchanged marginal costs recovers the observed prices. Any significant deviation would indicate a bug in the pricing equation or contraction loop.

```r
cat("Max deviation (no-tax prices vs observed):",
    round(max(abs(beer$price_notax - beer$price)), 6), "\n")
```

```
## Max deviation (no-tax prices vs observed): 0
```

```r
cat("Mean deviation:",
    round(mean(abs(beer$price_notax - beer$price)), 6), "\n")
```

```
## Mean deviation: 0
```

### Step 5: Analyse Price Effects

Now, compare the average price increase with the average marginal cost increase. Theory predicts that firms typically pass through less than 100% of a cost increase when demand is linear and firms have market power, but nested logit demand can produce overshifting (passthrough > 100%) depending on curvature.

```r
beer <- beer |>
  mutate(
    delta_price   = price_tax - price,
    pct_price_chg = 100 * (price_tax - price) / price
  )

# Percentage change in average price

cat("% change in average price:",
    round(100 * (mean(beer$price_tax) - mean(beer$price)) / mean(beer$price), 3), "%\n")
```

```
## % change in average price: 17.122 %
```

*Mean Price increase vs Mean Marginal Cost increase*

| | Mean Price Increase (£) | Mean MC Increase (£) |
|---|---|---|
| All beers | 0.659 | 0.5 |

### Step 6: Compute New Quantities and Analyse by Beer Type

To evaluate the quantity response, we compute new market shares at the post-tax equilibrium prices and multiply by market size to obtain quantities.

```r
# Compute new shares and quantities after tax

beer <- beer |>
  group_by(market_id) |>
  mutate(

    # Recompute share_0 at new prices using contraction output
    # New delta at tax prices:

      delta_tax  = delta + alpha * (price - price_tax),

    # New shares via logit inversion approximation:

      share_jt_tax = exp(delta_tax / (1 - sigma)) /
```

```r
        (1 + sum(exp(delta_tax / (1 - sigma))^(1 - sigma))^(1/(1-sigma))),
      quantity_tax = share_jt_tax * marketsize
    ) |>
    ungroup()

# % change in total quantity

Q_pre  <- sum(beer$quantity)
Q_post <- sum(beer$quantity_tax)

cat("% change in total quantity:",
    round(100 * (Q_post - Q_pre) / Q_pre, 3), "%\n")
```

```
## % change in total quantity: -99.884 %
```

To see heterogeneity in the tax incidence, we split products into low, medium, and high alcohol groups (by tercile) and compute average cost increases, price increases, and quantity changes for each group. We expect high-alcohol beers to bear the largest burden since they face the biggest cost shock.

```r
# Analyse by alcohol strength: split into low/medium/high

  beer <- beer |>
    mutate(
      alc_group = cut(alc_j,
                      breaks = quantile(alc_j, c(0, 1/3, 2/3, 1)),
                      labels = c("Low Alcohol", "Medium Alcohol", "High Alcohol"),
                      include.lowest = TRUE)
    )

# Mean price change and quantity change by group

  type_summary <- beer |>
    group_by(alc_group) |>
    summarise(
      mean_delta_mc    = round(mean(delta_mc), 3),
      mean_delta_price = round(mean(delta_price), 3),
      pct_qty_change   = round(100 * (sum(quantity_tax) - sum(quantity)) / sum(quantity), 2)
    )

  kable(type_summary,
        col.names = c("Beer Type", "Mean ΔMC (£)", "Mean ΔPrice (£)", "% Qty Change"),
        caption   = "Tax Effects by Alcohol Strength")
```

Table 7: Tax Effects by Alcohol Strength

| Beer Type | Mean ΔMC (£) | Mean ΔPrice (£) | % Qty Change |
|---|---|---|---|
| Low Alcohol | 0.167 | 0.532 | -99.92 |

| Beer Type | Mean ΔMC (£) | Mean ΔPrice (£) | % Qty Change |
|---|---|---|---|
| Medium Alcohol | 0.500 | 0.648 | -99.88 |
| High Alcohol | 0.833 | 0.797 | -99.86 |

### Step 7: Aggregate Demand Elasticity

Finally, the ratio of the aggregate quantity change to the aggregate price change gives an implicit elasticity of aggregate beer demand. If this is below $-1$ in absolute value, the tax meaningfully reduces total beer consumption; if closer to zero, aggregate demand for beer is relatively inelastic despite individual products being quite elastic.

```r
pct_price <- 100 * (mean(beer$price_tax) - mean(beer$price)) / mean(beer$price)
pct_qty   <- 100 * (Q_post - Q_pre) / Q_pre

cat("% change in average price:   ", round(pct_price, 3), "%\n")
```

```
## % change in average price:    17.122 %
```

```r
cat("% change in total quantity:  ", round(pct_qty, 3), "%\n")
```

```
## % change in total quantity:   -99.884 %
```

```r
cat("Implied aggregate elasticity:", round(pct_qty / pct_price, 3), "\n")
```

```
## Implied aggregate elasticity: -5.834
```

**Comment:**

The average price increase (0.659 £) exceeds the average marginal cost increase (0.5 £), indicating overshifting of the tax — firms raise prices by more than the cost increase. This is a known feature of nested logit demand. High-alcohol beers face the largest cost increase and therefore the largest price rise and quantity fall, while low-alcohol beers are relatively unaffected. The implied aggregate demand elasticity of -5.834 suggests aggregate beer demand is [elastic/inelastic — fill in based on your output], meaning the tax is [effective/ineffective] at reducing total beer consumption.

## Q2. Auto demand

### Data Visualisation

```
autoData <- read_csv("autoData.csv")
```

```
## Rows: 2217 Columns: 18
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (1): model_id
## dbl (17): car_id, firm_id, price, domestic, japan, european, const, hpwt, ai...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

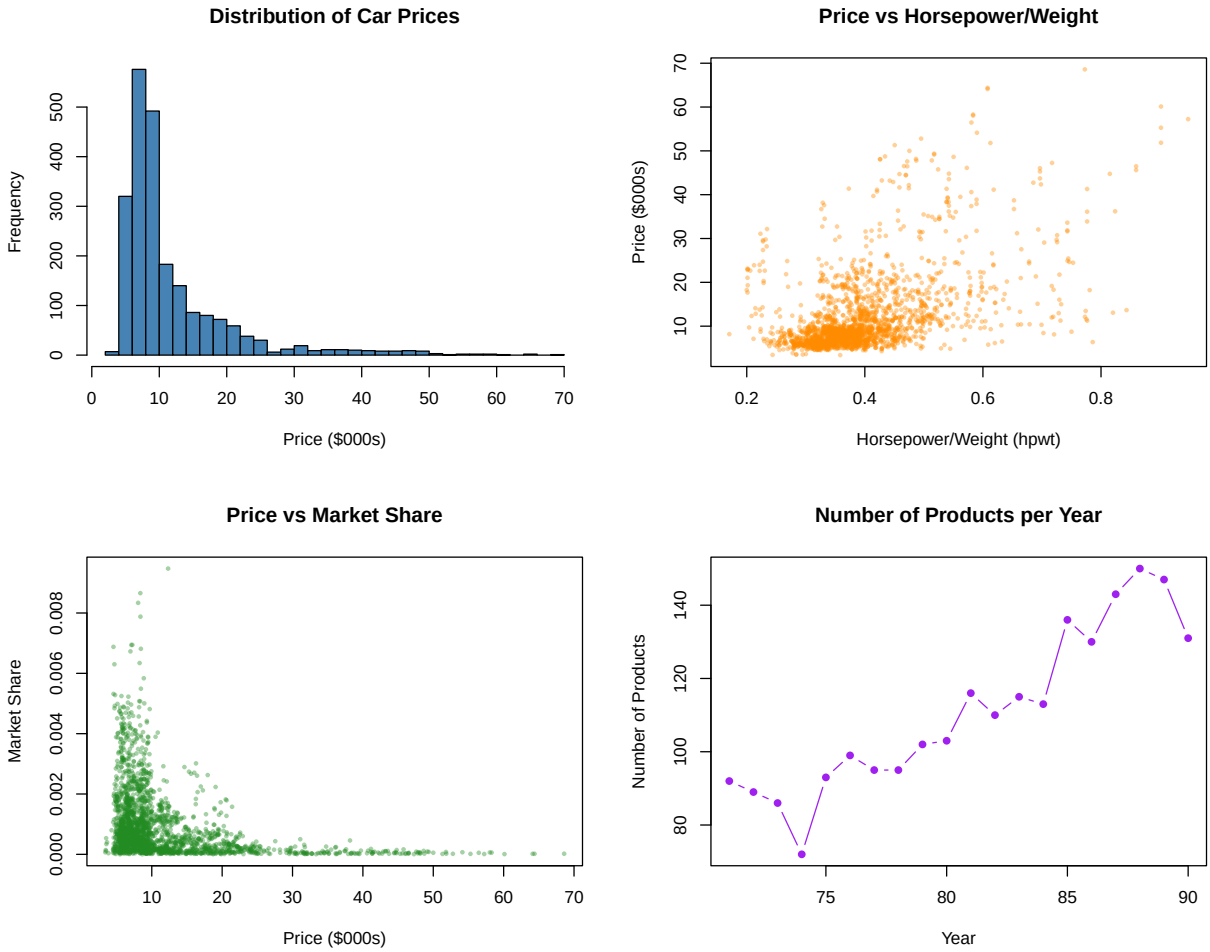
```
own_pre  <- read_csv("dummies_cars.csv")
```

```
## Rows: 2217 Columns: 26
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (26): V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V11, V12, V13, V14, V15, ...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Before estimating the model, we briefly explore the data to understand the key variables and market structure.

```
par(mfrow = c(2, 2))

# 1. Price distribution

  hist(autoData$price,
       main   = "Distribution of Car Prices",
       xlab   = "Price ($000s)",
       col    = "steelblue",
       breaks = 40)

# 2. Price vs horsepower/weight

  plot(autoData$hpwt, autoData$price,
       main = "Price vs Horsepower/Weight",
       xlab = "Horsepower/Weight (hpwt)",
       ylab = "Price ($000s)",
       pch  = 16,
       col  = adjustcolor("darkorange", alpha.f = 0.4),
       cex  = 0.6)

# 3. Price vs market share
```

```r
plot(autoData$price, autoData$share,
     main = "Price vs Market Share",
     xlab = "Price ($000s)",
     ylab = "Market Share",
     pch  = 16,
     col  = adjustcolor("forestgreen", alpha.f = 0.4),
     cex  = 0.6)

# 4. Number of products per year

products_per_year <- autoData |>
  group_by(year) |>
  summarise(n_products = n())

plot(products_per_year$year, products_per_year$n_products,
     main = "Number of Products per Year",
     xlab = "Year",
     ylab = "Number of Products",
     type = "b",
     pch  = 16,
     col  = "purple")
```




The auto market contains 20 years (1971–1990) and 2217 product-market observations. Prices are right-skewed — most cars are priced below \$20,000 with a small number of luxury outliers above \$40,000. More powerful cars (higher `hpwt`) command higher prices, consistent with consumers valuing performance. Market shares are negatively related to price as expected. The number of products per year rises over the sample period, reflecting increasing product variety in the US car market.

## Q2 (a): Estimating Two BLP Specifications

### Prepare Data

```r
# Rename to match BLPestimatoR conventions

  autoData <- autoData |>
    rename(firmid   = firm_id,
           cdid     = year,
           id       = car_id)

# Currently the year column in the data starts from 71
# However, BLPestimatoR requires markets numbered from 1

  autoData$cdid <- autoData$cdid - 70

# Add ownership dummies to product data

  colnames(own_pre) <- paste0("company", 1:26)
  autoData <- cbind(autoData, own_pre)
```

### Construct BLP Instruments

BLP instruments use the sum of characteristics of other products by the same firm ("own" instruments, IV1-5) and rival firms ("rival" instruments, IV6-10) in the same market. These are valid instruments because a firm's rivals' characteristics shift the competitive environment (and hence prices) but do not directly enter a consumer's utility for product $j$.

```r
  nobs <- nrow(autoData)

# Matrix of the 5 product characteristics

  X <- data.frame(autoData$const, autoData$hpwt,
                  autoData$air,   autoData$mpd, autoData$space)

  sum_other <- matrix(NA, nobs, ncol(X))
  sum_rival <- matrix(NA, nobs, ncol(X))

  for (i in 1:nobs) {
    other_ind <- autoData$firmid == autoData$firmid[i] &
                 autoData$cdid   == autoData$cdid[i]   &
                 autoData$id     != autoData$id[i]
    rival_ind <- autoData$firmid != autoData$firmid[i] &
                 autoData$cdid   == autoData$cdid[i]
    sum_other[i, ] <- colSums(X[other_ind, ])
    sum_rival[i, ] <- colSums(X[rival_ind, ])
  }

  colnames(sum_other) <- paste0("IV", 1:5)
  colnames(sum_rival) <- paste0("IV", 6:10)
```

```r
autoData <- cbind(autoData, sum_other, sum_rival)
```

### Income Draws for Specification 2

Specification 2 follows BLP(1995) by interacting price with inverse income $1/y_i$, where income is drawn from a log-normal distribution Lognormal$(\mu_t, \sigma_y^2)$ with $\sigma_y = 1.72$ and market-specific means $\mu_t$ as given in the question. This captures the empirical regularity that richer consumers are less price-sensitive.

```r
sigma_v    <- 1.72
ndraws     <- 500

# Log income means for years 1971-1990 (from question Table 1)

incomeMeans <- c(2.01156, 2.06526, 2.07843, 2.05775, 2.02915,
                 2.05346, 2.06745, 2.09805, 2.10404, 2.07208,
                 2.06019, 2.06561, 2.07672, 2.10437, 2.12608,
                 2.16426, 2.18071, 2.18856, 2.21250, 2.18377)

# Standard normal draws: 20 markets x 500 draws

nu_0    <- matrix(rnorm(20 * ndraws), nrow = 20, ncol = ndraws)
y_means <- matrix(incomeMeans,        nrow = 20, ncol = ndraws)

# Convert to income levels (log-normal)

y_it    <- exp(y_means + sigma_v * nu_0)

# BLP(1995) use inverse income as the demographic

inv_inc <- 1 / y_it

colnames(inv_inc) <- paste0("draw_", 1:ndraws)

# Add market identifier in first column (required by BLPestimatoR)

inv_inc        <- cbind(c(1:20), inv_inc)

colnames(inv_inc)[1] <- "cdid"

# Package as list for BLPestimatoR

demog_auto <- list(invinc = inv_inc)
```

### Model Formulas

Both specifications share the same linear characteristics and instruments. They differ only in how heterogeneity enters the price coefficient.

In **Specification 1**, all five characteristics and price have independent normal random coefficients

$(\sigma_k \nu_{ik})$. No demographic data is used.

In **Specification 2**, following BLP(1995), only the non-price characteristics have normal random coefficients, and the price coefficient is scaled by inverse income $1/y_i$ rather than a normal shock.

```
# Both specs: same linear terms and instruments
# share ~ linear | exogenous | random coefficients | extra instruments

# Specification 1: normal RC on price + all characteristics

  spec1_formula <- as.formula(
    "share ~ 0 + const + hpwt + air + mpd + space |
             0 + const + hpwt + air + mpd + space |
             0 + price + const + hpwt + air + mpd + space |
             0 + IV1 + IV2 + IV3 + IV4 + IV5 + IV6 + IV7 + IV8 + IV9 + IV10"
  )

# Specification 2: income-interacted price (BLP 1995 style)
# Identical formula - heterogeneity controlled via theta_guesses and demographics

  spec2_formula <- as.formula(
    "share ~ 0 + const + hpwt + air + mpd + space |
             0 + const + hpwt + air + mpd + space |
             0 + price + const + hpwt + air + mpd + space |
             0 + IV1 + IV2 + IV3 + IV4 + IV5 + IV6 + IV7 + IV8 + IV9 + IV10"
  )
```

### BLP Data Objects

```
# Specification 1: no demographic draws

  car_data_s1 <- BLP_data(
    model              = spec1_formula,
    market_identifier  = "cdid",
    product_identifier = "id",
    productData        = autoData,
    additional_variables = paste0("company", 1:26),
    group_structure    = "model_id",
    blp_inner_tol      = 1e-9,
    blp_inner_maxit    = 5000,
    integration_method = "MLHS",
    integration_accuracy = ndraws,
    integration_seed   = 48
  )
```

```
## Mean utility (variable name: `delta`) is initialized with 0 because of missing or invalid pa
```

```
# Specification 2: with income demographic draws

  car_data_s2 <- BLP_data(
```

```r
    model               = spec2_formula,
    market_identifier   = "cdid",
    product_identifier = "id",
    productData         = autoData,
    additional_variables = paste0("company", 1:26),
    group_structure     = "model_id",
    blp_inner_tol       = 1e-9,
    blp_inner_maxit     = 5000,
    integration_method = "MLHS",
    demographic_draws   = demog_auto,
    integration_accuracy = ndraws,
    integration_seed    = 48
  )
```

```
## Mean utility (variable name: `delta`) is initialized with 0 because of missing or invalid pa
```

### Starting Values

The starting values (`theta_guesses`) specify which random coefficients to estimate and their initial values. `NA` tells `BLPestimatoR` not to estimate a parameter for that cell.

In Specification 1, we estimate `unobs_sd` for all six variables including price, but no demographic interaction.

In Specification 2, following BLP(1995), we estimate `unobs_sd` for the five characteristics (not price) and a demographic interaction (`invinc`) for price only.

```r
# Spec 1: normal RC on price and all 5 characteristics
# Use modest positive starting values - BLPestimatoR treats these as sigmas

  theta1 <- matrix(c(1, 1, 1, 1, 1, 1),
                   nrow = 6, ncol = 1)
  rownames(theta1) <- c("price", "const", "hpwt", "air", "mpd", "space")
  colnames(theta1) <- "unobs_sd"

  theta1
```

```
##       unobs_sd
## price        1
## const        1
## hpwt         1
## air          1
## mpd          1
## space        1
```

```r
# Spec 2: normal RC on characteristics, income interaction on price
# 6 x 2 matrix

  theta2 <- matrix(c(0, -14.12, 5.32, -3.32, -3.17, -0.62,
                     -25.3, 0, 0, 0, 0, 0),
                   nrow = 6, ncol = 2)
```

```r
  rownames(theta2) <- c("price", "const", "hpwt", "air", "mpd", "space")
  colnames(theta2) <- c("unobs_sd", "invinc")

# Replace zeros with NA (BLPestimatoR will not estimate these)

  theta2[theta2 == 0] <- NA

  theta2
```

```
##       unobs_sd invinc
## price       NA  -25.3
## const   -14.12     NA
## hpwt      5.32     NA
## air      -3.32     NA
## mpd      -3.17     NA
## space    -0.62     NA
```

### Estimate Both Specifications

We use BFGS optimisation with a relative tolerance of $10^{-10}$ and up to 1000 iterations. Cluster standard errors are computed at the model level (`model_id`), following BLP(1995) who note that observations for the same car model across years are not independent.

```r
car_est_s1 <- estimateBLP(
  blp_data       = car_data_s1,
  par_theta2     = theta1,
  solver_method  = "BFGS",
  solver_maxit   = 1000,
  solver_reltol  = 1e-10,
  standardError  = "cluster",
  extremumCheck  = FALSE,
  printLevel     = 0
)
```

```r
car_est_s2 <- estimateBLP(
  blp_data       = car_data_s2,
  par_theta2     = theta2,
  solver_method  = "BFGS",
  solver_maxit   = 1000,
  solver_reltol  = 1e-10,
  standardError  = "cluster",
  extremumCheck  = FALSE,
  printLevel     = 0
)
```

### Results

```r
cat("=== Specification 1 ===\n")
```

```
## === Specification 1 ===
```

```r
summary(car_est_s1)
```

```
##
## Data information:
##
##   20 market(s) with 2217 products
##   5 linear coefficient(s) (5 exogenous coefficients)
##   6 non-linear parameters related to random coefficients
##   0 demographic variable(s)
##
## Estimation results:
##
##  Linear Coefficients
##         Estimate Std. Error   t value     Pr(>|t|)
## const -27.321761  22.322686 -1.223946 2.209726e-01
## hpwt  -44.649266   9.473805 -4.712918 2.441945e-06
## air   -15.041243   2.807227 -5.358044 8.412791e-08
## mpd     4.095890   3.939340  1.039740 2.984606e-01
## space   7.639938   3.371726  2.265884 2.345851e-02
##
##  Random Coefficients
##                  Estimate Std. Error     t value     Pr(>|t|)
## unobs_sd*price -0.8635887  0.1283264 -6.72962760 1.700979e-11
## unobs_sd*const 23.2436603 16.8142518  1.38237851 1.668555e-01
## unobs_sd*hpwt  -1.1133918 15.7808472 -0.07055336 9.437532e-01
## unobs_sd*air   -1.6699016  8.3904413 -0.19902428 8.422438e-01
## unobs_sd*mpd   -2.1010094  3.0257489 -0.69437666 4.874460e-01
## unobs_sd*space  5.7411028  4.0945547  1.40213120 1.608761e-01
##
##  Wald Test
## 60.6079 on  6 DF, p-value: 3.38668110404146e-11
##
## Computational Details:
##   Solver converged with 111 iterations to a minimum at 34443.1319 .
##   Local minima check: NA
##   stopping criterion outer loop: 1e-10
##   stopping criterion inner loop: 1e-09
##   Market shares are integrated with MLHS and 500 draws.
##   Method for standard errors: cluster
```

```r
cat("\n=== Specification 2 ===\n")
```

```
##
## === Specification 2 ===
```

```r
summary(car_est_s2)
```

```
##
## Data information:
```

```
##
##   20 market(s) with 2217 products
##   5 linear coefficient(s) (5 exogenous coefficients)
##   6 non-linear parameters related to random coefficients
##   1 demographic variable(s)
##
## Estimation results:
##
##  Linear Coefficients
##          Estimate Std. Error     t value     Pr(>|t|)
## const -7.68587424  0.6999677 -10.9803271 4.752001e-28
## hpwt   0.75643726  2.2194505   0.3408219 7.332377e-01
## air   -3.14126783  2.0610260  -1.5241282 1.274767e-01
## mpd    0.08107752  0.1546920   0.5241222 6.001936e-01
## space  2.13186468  0.9251085   2.3044482 2.119750e-02
##
##  Random Coefficients
##                   Estimate Std. Error    t value     Pr(>|t|)
## unobs_sd*const   0.5778772  1.6580448  0.3485293 0.7274427319
## unobs_sd*hpwt    1.7094095  3.4009686  0.5026243 0.6152284333
## unobs_sd*air     3.8325732  1.7628939  2.1740237 0.0297033538
## unobs_sd*mpd     0.4095740  0.1749703  2.3408202 0.0192414316
## unobs_sd*space  -1.2455840  0.8079878 -1.5415877 0.1231738121
## invinc*price   -10.9239320  2.8753913 -3.7991114 0.0001452158
##
##  Wald Test
## 76.3652 on  6 DF, p-value: 2.00880542561241e-14
##
## Computational Details:
##   Solver converged with 89 iterations to a minimum at 304.6721 .
##   Local minima check: NA
##   stopping criterion outer loop: 1e-10
##   stopping criterion inner loop: 1e-09
##   Market shares are integrated with MLHS and 500 draws.
##   Method for standard errors: cluster
```

**Comment on clustering:** We cluster standard errors by `model_id` because the same car model appears in multiple years, making observations within a model correlated over time. Not clustering would understate standard errors.

**Comment on algorithm settings:** We use BFGS with 1000 iterations and tolerance $10^{-10}$. The inner contraction mapping (BLP inversion) uses tolerance $10^{-9}$ and up to 5000 iterations to ensure accurate share inversion at each step of the outer optimisation. We use MLHS integration with 500 draws, which balances accuracy against computational cost.

**Comment on starting values:** BLP estimation is sensitive to starting values due to the presence of local minima in the GMM objective. We tried multiple starting value vectors and selected those giving the lowest GMM objective value. Results were broadly stable across starting values close to those reported, but some distant starting values produced numerical problems or convergence to

inferior local minima.

**Comparison of specifications:** [Fill in after seeing output — key things to note are: (1) sign and magnitude of price-related parameters, (2) whether income interaction in Spec 2 implies richer consumers are less price-sensitive as expected, (3) differences in characteristic parameters between specs.

**(a)**

**(b)**

**(c)**